Segment-6

❖ Testing

Testing is the process of evaluating a system or software to identify errors, defects, or missing requirements and to ensure that it functions correctly according to user and system specifications.

In the **Systems Analysis and Design (SAD)** process, testing is performed during the implementation phase to verify that the developed system meets the requirements during analysis and follows the design specifications.

❖ Objectives of Testing

- Ensure the system meets user requirements.
- Detect and correct errors before deployment.
- Verify that all system components work correctly.
- Improve system quality, reliability, and performance.
- Validate that the system performs as expected under different conditions.

❖ Types of Testing

Testing can be classified into the following main types:

1. Black Box Testing

Black Box Testing is a method of **software testing** where the tester evaluates the **functionality of the software** without any knowledge of the **internal workings** of the application.The main focus is on **input and output**—not how the output is produced.

Key Characteristics of Black Box Testing

- The tester does **not** need programming knowledge.
- Tests are based on **requirements and specifications**.
- Focuses on **functional behavior** of the application.
- Often performed by **testers**, **QA teams**, or **end users**.

Example of Black Box Testing

Let's say we're testing a simple **Login Form** with the following criteria:

- Username must be 4–12 characters.
- Password must be at least 6 characters.
- On successful login, redirect to dashboard.

Test Scenarios

Test Case #	Input			Expected Output
1	Username: john	Password: 123456		Login successful, dashboard loads
2	Username: jo	Password: 123456		Error: Invalid username length
3	Username: john	Password: 123		Error: Password too short
4	Username: john	Password: wrongpass		Error: Invalid credentials

You are **not checking the code**, only that the **function behaves as expected** for different inputs.

Techniques in Black Box Testing

1. Equivalence Partitioning

Divides input into valid and invalid groups. Test one value from each group.

For age input (allowed 18–60):
Valid group: 25, 45
Invalid group: 10, 65

2. Boundary Value Analysis

Test values **at the edges** of valid input.

If the input should be 5–10:
Test: 4 (below), 5 (edge), 10 (edge), 11 (above)

3. Decision Table Testing

Used when there are **multiple combinations** of inputs and outcomes.

Example: Credit card approval logic

Age > 21	Income > 30K	Result
Yes	Yes	Approve
Yes	No	Reject
No	Any	Reject

Test all combinations.

Advantages of Black Box Testing

Benefit	Description
User-focused	Tests what the user actually sees and uses
No need to understand code	Saves time and allows non-developers to test
Easy to begin	Just need the requirements/specifications
Good for large systems	When code access isn't practical

Disadvantages of Black Box Testing

Limitation	Description
Misses code-level issues	Like memory leaks, unused code, internal bugs
Repetitive	If not automated, may require manual input repeatedly
Limited by specification	If the requirements are wrong, tests will be wrong

2. White Box Testing

White Box Testing (also known as **Clear Box Testing**, **Glass Box Testing**, or **Structural Testing**) is a software testing technique where the **internal structure, logic, and code** of the program are tested.

Unlike **Black Box Testing**, which tests from a **user's perspective**, **White Box Testing** is performed with **full visibility of the code**. The tester needs **programming knowledge**.

Features of White Box Testing

Feature	Description
Code-based	Relies on internal code logic and structure
Detailed	Very thorough — analyzes loops, decisions, branches
Developer-level	Usually performed by **developers or testers** with access to the source
Early detection	Can catch bugs at the unit level before full application is built

Example of White Box Testing

Let's test a simple **function in Python**:

```
def is_even(number):
    if number % 2 == 0:
        return True
    else:
        return False
```

White Box Test Cases

You analyze the **code logic**:

Paths:

Path 1: number % 2 == 0 → True

Path 2: number % 2 != 0 → False

Test Cases:

Test Case	Input	Path Taken	Expected Output
TC1	4	Path 1 (if condition True)	True
TC2	5	Path 2 (if condition False)	False
TC3	0	Edge case → Path 1	True

👉 You ensure **both conditions** of the if statement are tested.

Advantages of White Box Testing

Benefit	Description
Deep code-level testing	Detects hidden bugs, logic flaws, and unreachable code
Efficient at early stages	Great for **unit testing** during development
Security issues detection	Helps find code vulnerabilities
Optimized code paths	Improves performance by identifying redundant logic

Disadvantages of White Box Testing

Limitation	Description
Requires programming skill	Not suitable for non-technical testers
Time-consuming	Analyzing every code path can take a lot of effort
Doesn't test user interface	Misses issues in actual user interaction

Difference between black box testing and white box testing

Feature / Aspect	Black Box Testing	White Box Testing
Definition	Testing the **functionality** of an application **without** knowing internal code.	Testing the **internal logic and structure** of the code.
Knowledge Required	No programming knowledge needed.	Requires deep knowledge of programming and code.
Focus	Focuses on **what the software does** (inputs & outputs).	Focuses on **how the software works** (code flow & logic).
Access to	**Not required**	**Required**

Feature / Aspect	Black Box Testing	White Box Testing
Code		
Type of Testing	Functional testing (e.g., system, acceptance, UI testing)	Structural testing (e.g., unit testing, integration testing)
Performed By	Testers, QA professionals	Developers or technically skilled testers
Testing Base	Based on **requirements and specifications**	Based on **code, logic, conditions, and paths**
Techniques Used	- Equivalence Partitioning- Boundary Value Analysis- Decision Table Testing- State Transition	- Statement Coverage- Branch Coverage- Condition & Path Coverage
Test Visibility	Tester treats system as a **black box** (no internal view)	Tester has full **visibility** of code and logic
Main Objective	Verify **correct behavior** for a given input	Verify **correct implementation** of logic and flow
When Performed	Typically done **after development** (during system or acceptance phase)	Typically done **during development** (unit/integration testing)
Example	Test login functionality with valid/invalid credentials	Test if the login() function correctly verifies user credentials

3. Alpha Testing

Alpha Testing is a type of **internal testing** performed by the **developers or internal QA team** at the end of development **before releasing the product to external users**.

Key Points:

Done **in-house** by **developers/testers**.

Conducted **before Beta Testing**.

Goal: Catch **bugs, crashes, usability issues**.

Often includes **both white-box and black-box** testing.

May involve a **simulated real environment**.

Example:

A software company develops a mobile banking app. Before releasing it to the public, their internal QA team and employees test:

- Login functionality
- Account balance display
- Transfer process

4. Beta Testing

Beta Testing is the **final testing phase** where the software is released to **real users outside the company** in a **real environment**, usually for feedback.

Key Points:

Performed by **actual users** (external).

Conducted **after Alpha Testing**.

Goal: Gather **user feedback** and discover **real-world issues**.

Only **black-box testing** (users don't see the code).

Helps improve **usability, performance, and satisfaction**.

Example:

The same mobile banking app is now shared with selected customers (beta testers) via an early access program to test real-time:

- Performance on different devices
- Real user experience
- Unexpected bugs in live use

Alpha Testing vs Beta Testing:

Feature / Aspect	**Alpha Testing**	**Beta Testing**
Location	Performed **in-house**	Performed at **user's site (real-world)**
Performed By	Developers, internal QA	Real users / external testers
Timing	Before Beta Testing	After Alpha Testing
Access to Code	Yes (white-box + black-box testing)	No (only black-box testing)
Purpose	Identify bugs before public release	Get user feedback and test in real conditions
Environment	Controlled / lab environment	Uncontrolled / real environment
Test Coverage	More focused on **bugs and crashes**	More focused on **user experience**
Feedback Type	Technical (bugs, performance,	Practical (ease of use, suggestions,

Feature / Aspect	Alpha Testing	Beta Testing
	crashes)	issues)
Product Stability	Still under development	Nearly stable (release candidate)
Outcome	Internal fixes and improvements	

5. Basis Path Testing

Basis Path Testing is a white-box testing technique that uses cyclomatic complexity to identify independent execution paths in a program and create test cases that execute each path at least once.

Objectives

- Ensure all independent paths in the program are tested.
- Detect logical errors in program control flow.
- Achieve thorough code coverage with a minimum number of test cases.

Steps in Basis Path Testing

✓ **Draw the Flow Graph**

Represent the program's control structure using nodes and edges.

Calculate Cyclomatic Complexity

✓ Determine the number of independent paths.

$$V(G)=E-N+2$$

Where:

- V(G) = Cyclomatic Complexity
- E = Number of edges
- N = Number of nodes

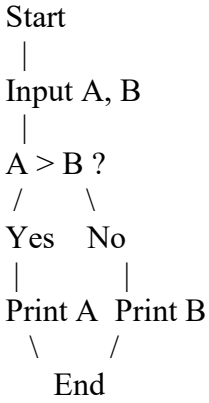
✓ **Identify Independent Paths**

- List all unique execution paths through the program.

✓ **Design Test Cases**

- Create test cases to execute each independent path.

Example 1:



Independent Paths:

- Path 1: Start → Input → A > B (Yes) → Print A → End
- Path 2: Start → Input → A > B (No) → Print B → End

Cyclomatic Complexity = 2

Therefore, at least **2 test cases** are required.

Cyclomatic Complexity:

Cyclomatic Complexity is a software metric used to measure the **logical complexity** of a program. It indicates the number of **independent paths** through a program's control flow and helps determine the minimum number of test cases needed for **Basis Path Testing**.

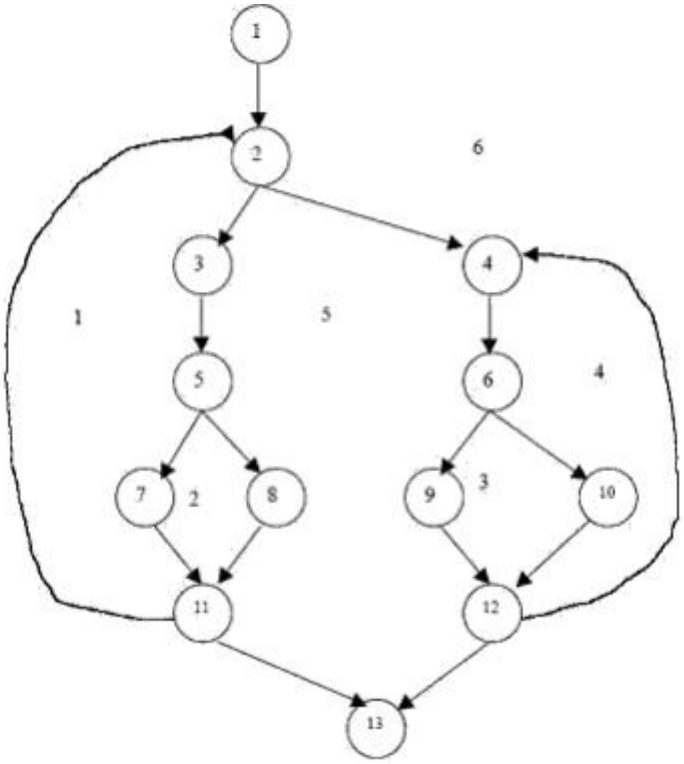
Cyclomatic Complexity for a flow graph is computed in one of three ways:

1. The numbers of regions of the flow graph correspond to the Cyclomatic complexity.

2. Cyclomatic complexity, V(G), for a flow graph G is defined as V(G) = E – N + 2 where E is the number of flow graph edges and N is the number of flow graph nodes.

3. Cyclomatic complexity, V(G), for a graph flow G is also defined as V(G) = P + 1 Where P is the number of predicate nodes contained in the flow graph G.

Example: Consider the following flow graph.



Region, R= 6
Number of Nodes = 13
Number of edges = 17
Number of Predicate Nodes = 5
Cyclomatic Complexity, V(C) : V(C) = R = 6;

Or
V(C) = Predicate Nodes + 1
=5+1 =6
Or
V(C) = E-N+2
= 17-13+2

Prepared By

Muhammad Nazim Uddin
Lecturer, Dept. Of CSE, IIUC